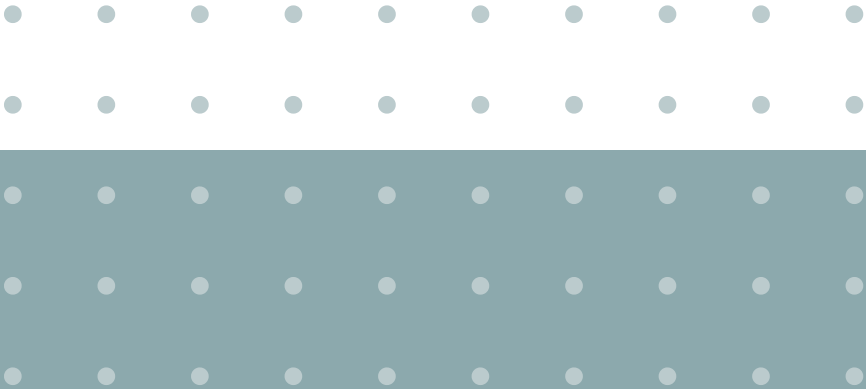




Pricing and Regional Trends in Airbnb Markets



Team A
Limeng Jiang
Jiawen Yin
Kanak Kedar Kale
Omkar Shashank Pathare

PROBLEM STATEMENT & OBJECTIVE



- The goal of this project is to analyze Airbnb listings across major U.S. cities to understand the key factors influencing pricing, ratings, and demand. We will study variables such as property characteristics and host quality to identify patterns that drive listing prices and guest satisfaction. We will also compare pricing and occupancy trends between the East and West Coasts to find regional differences in customer preferences and market behavior.
- Questions to explore:
 1. What factors determine Airbnb listing prices?
e.g., location, amenities, host rating, etc.
 2. How is demand affected by price changes?
Measure how price changes affect occupancy rate.
 3. What are the trends across the East Coast and the West Coast?
Explore aspects such as how prices differ in cities and when the occupancy rate is the highest.

DATA SOURCE AND DESCRIPTION

Source:

- Data obtained from: <https://insideairbnb.com/get-the-data/>

Description:

- The dataset was obtained from Inside Airbnb, which provides publicly available data on Airbnb listings.
- It includes listings.csv, calendar.csv, reviews.csv, and hosts.csv files with information on prices, availability, reviews, and host details.
- Data were collected for selected East Coast and West Coast cities in the United States.
- These data support the analysis of pricing factors, ratings, and regional market trends.

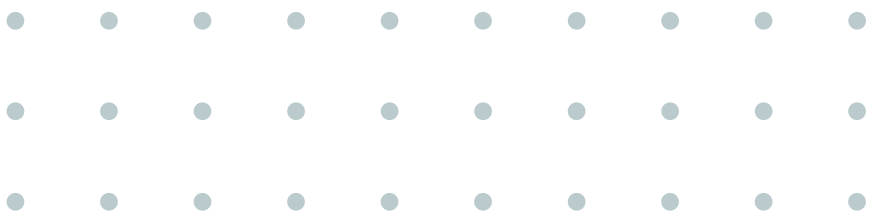
City Selection:

East Coast

- New York City, NY
- Boston, MA
- Washington, D.C.
- Miami, FL

West Coast

- Los Angeles, CA
- San Francisco, CA
- Seattle, WA
- San Diego, CA



QUESTION 1: WHAT FACTORS DETERMINE AIRBNB PRICES

Section 1. Explore which factors have an influence on the Airbnb listing prices

- Data preprocessing:
 - Preprocess listings.csv: convert dtypes, fill missing values with mode/mean/median, one-hot encode categorical features
 - Preprocess calendar.csv: convert date and price formats, clean invalid records, aggregate daily data to weekly level, and generate modeling-ready features
- Data analysis:
 - Categorical features: room type, property type, neighborhood, instant bookable, superhost status - ANOVA
 - Numerical features: price, booked_ratio, review_scores_rating, amenities_count, demand_score

Section 2. Build regression models to predict Airbnb listing price

- **Linear regression (lr):** baseline linear model + interpretable coefficients for numeric features + trained with regularization to improve stability and reduce overfitting
- **Gradient Boosting regression (GBT):** non-linear regression model + performs well with mixed numerical and categorical data + trained with regularization

SECTION 1.DATA PREPROCESSING & ANALYSIS

listings.csv:

- **Feature dtype:** convert strings into numeric feature, ensuring that all numeric fields are properly casted
- **Missing value handling:** parsed structured fields (e.g., bathrooms from bathrooms_text), applied mode/median/mean imputation for numeric variables
- **Categorical processing:** added explicit “missing” categories and applied one-hot encoding to categorical variables for compatibility with tree-based models (GBT)

calendar.csv:

- **Converted** date to DateType and price to DoubleType
- **Standardized** price by removing currency symbols and commas
- **Handled** missing values by dropping invalid records and filling remaining values at the weekly level
- **Aggregated** daily observations to weekly-level data
- **Computed** availability_rate as the weekly availability metric
- **Engineered** features including quarter (one-hot encoded), trend_yearly, price_per_booked_day, and demand_score = booked_ratio × avg_price
- **Joined** the processed calendar features with listings data

SECTION 2. BUILD A LINEAR REGRESSION MODEL

Feature selection

- Used a **correlation matrix** to remove highly correlated features
- Applied a **low-variance filter** to drop features with little variation
- Combined both methods to obtain the **final selected features**: ['property_type_index', 'review_scores_accuracy_clean', 'accommodates_clean', 'bedrooms_clean', 'bathrooms_clean', 'review_scores_rating_clean', 'room_type_index', 'booked_ratio', 'beds_clean', 'amenities_count']

Linear Regression model

- Built a Linear Regression model to predict Airbnb New York City price using the selected features
- Used Elastic Net to reduce overfitting and improve model stability
- Scaling Up: Trained and evaluated the model on different sample sizes
- Scaling Out: Trained in a single node cluster and 1 master with 2 worker nodes cluster

Data Percentage	Single Node Training Time	2 Worker Nodes Training Time	Time Reduction Percentage	RMSE
10%	27.71	20.29	26.77%	493.73
25%	10.67	4.63	56.61%	667.57
50%	25.1	5.37	78.59%	699.68
100%	43.89	7.72	82.42%	625.72

SECTION 2. BUILD A GBT MODEL

- ANOVA result: room type, property type
- Training & Validation: split the dataset into training (80%) and test (20%) sets to evaluate model generalization performance
- Scalability: trained the GBT model on increasing fractions of the data (10%, 25%, 50%, and 100%) to analyze performance gains and computational cost
- Model evaluation: assessed predictive accuracy using RMSE and R^2

Percent of Data	RMSE	R^2
10%	603.72	0.2852
25%	571.61	0.3894
50%	550.09	0.4345
100%	539.29	0.4565

- Monatomic improvement with increasing data
- Performance increase ceiling
- Noise-limited prediction

QUESTION2.

PRICE ELASTICITY VS DEMAND

How Is Demand Affected by Price Changes?

Understanding how price changes influence demand (occupancy rate) is a core economic and business problem. Classical economic theory predicts an inverse relationship between price and demand, captured through price elasticity.

However, real-world marketplace data is large, noisy, and heterogeneous, often producing unstable elasticity estimates when analyzed on small samples.

Demand response to price varies across:

Time (seasonality)

Location (city effects)

Product characteristics (room type, listing attributes)

Big Data Solution

To accurately measure price–demand relationships, we must analyze millions of observations, not isolated samples.

Big data enables:

- More stable elasticity estimation
- Validation of economic theory under real-world conditions

The problem is not only predictive, but explanatory:

- How sensitive is demand to price?
- Does this sensitivity remain stable across samples and clusters?

Formal Problem Definition (Technical View)

Objective: Measure the impact of price changes on demand (occupancy rate)

Primary quantity of interest:

- Price Elasticity of Demand (β)

Modeling approach:

- Log–log regression (elasticity interpretation)
- Optional non-linear models (RF / GBT) for robustness checks

Why This Problem Matters ?

Incorrect elasticity estimates can lead to:

Poor pricing decisions

Revenue loss

Misallocation of capacity

A scalable, interpretable solution allows:

Data-driven pricing strategies

Economically sound decision-making

Validation of the law of demand using big data

ETL PROCESS



Raw Data

Each record represents: A listing, A date, The listed price, Availability status (used to derive demand / occupancy)



ETL & Feature Engineering

Raw Airbnb calendar data contains millions of daily price-demand records. Data is cleaned by removing invalid prices, missing values, and outliers. Log transformations create log_price and log_demand for elasticity modeling. Cleaned and transformed data is cached for efficient, consistent reuse



Cached Dataset for Efficient Reuse

Cleaned and transformed Airbnb data is cached in Spark. Caching avoids repeated ETL computations. Enables efficient sampling and repeated model training. Ensures consistent results across experiments and clusters.



Cached Dataset for Efficient Reuse

Cached dataset serves as the single source of truth. Same dataset used for all downstream analysis. Supports sampling at multiple levels (5%, 15%, 40%, 100%). Used consistently for elasticity regression. Enables fair runtime and scalability comparison. After this stage, only the model changes — not the data

```
calendar_df = (
    calendar_raw
    .withColumn("price_str", F.regexp_replace("price", "[$,]", ""))
    .withColumn(
        "price_str",
        F.when(F.col("price_str").rlike("^[0-9.]+$"), F.col("price_str"))
        .otherwise(None)
    )
    .withColumn("price_clean", F.col("price_str").cast("double"))
    .filter("price_clean IS NOT NULL AND price_clean > 10 AND price_clean < 2000")
    .withColumn("booked", (1 - (F.col("available") == "t").cast("int")))
)
```

```
daily_df = (
    daily_df
    .withColumn("log_price", F.log(F.col("price_clean")))
    .withColumn("log_occ", F.log(F.col("booked") + 0.05))
)
```

```
fe_df = (
    daily_df.groupBy("listing_id")
    .agg(
        F.avg("log_price").alias("avg_log_price"),
        F.avg("log_occ").alias("avg_log_occ")
    )
)
```

```
model_df = (
    daily_df.join(fe_df, "listing_id")
    .withColumn("log_price_dm", F.col("log_price") - F.col("avg_log_price"))
    .withColumn("log_occ_dm", F.col("log_occ") - F.col("avg_log_occ"))
)
```

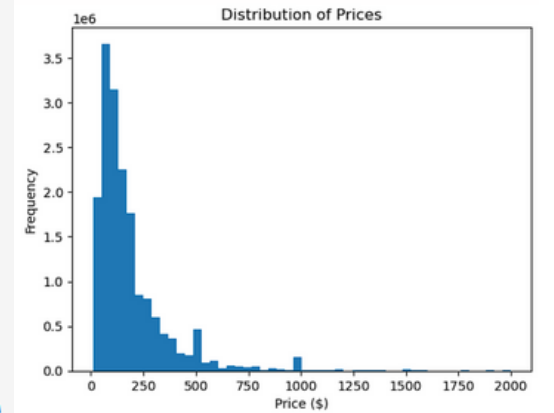
```
model_df = model_df.filter("log_price_dm != 0")
model_df = model_df.filter("log_occ_dm != 0")
```

```
sample_sizes = [0.05, 0.15, 0.40, 1.0]
sampled = {}
```

```
for s in sample_sizes:
    sampled[s] = model_df.sample(False, s, seed=42)
    print(f"{s*100:.0f}% sample rows:", sampled[s].count())
```

```
listing_df = (
    listing_raw
    .select(
        "id",
        "room_type",
        "review_scores_rating",
        "accommodates"
    )
    .withColumnRenamed("id", "listing_id")
)
```

```
daily_df = (
    airbnb
    .select(
        "listing_id",
        "date",
        "price_clean",
        "booked"
    )
    .filter("price_clean > 0")
)
```



MODEL TRAINING

Modeled price–demand relationship using log–log linear regression $\log(\text{Demand}) = \alpha + \beta \log(\text{Price}) + \varepsilon$

Used log(demand) as target and log(price) as predictor

Cached cleaned dataset in Spark for consistent reuse

Trained model using Spark MLlib distributed regression

Evaluated robustness as using 5%, 15%, 40%, 100% random samples

Fixed random seed ensured reproducibility across runs
Same pipeline used across all clusters and samples

```
def run_elasticity(df):  
  
    # Use log_price and log_occ directly (no FE version)  
    assembler = VectorAssembler(  
        inputCols=["log_price"],  
        outputCol="features"  
    )  
  
    train_df = assembler.transform(df).select("features", "log_occ")  
  
    lr = LinearRegression(  
        labelCol="log_occ",  
        regParam=0.01,          # stabilizes regression  
        elasticNetParam=0.0  
    )  
  
    lr_model = lr.fit(train_df)  
  
    return float(lr_model.coefficients[0])
```

```
import time  
  
elasticity_results = {}  
time_results = {}  
  
for s, df_sample in sampled.items():  
    start_time = time.perf_counter()  
  
    beta = run_elasticity(df_sample)  
  
    # Force Spark execution (important due to lazy evaluation)  
    df_sample.count()  
  
    end_time = time.perf_counter()  
  
    elasticity_results[s] = beta  
    time_results[s] = end_time - start_time  
  
    print(  
        f"Elasticity @ {s*100:.0f}% sample = {beta:.6f} | "  
        f"Time taken = {time_results[s]:.2f} seconds"  
    )
```

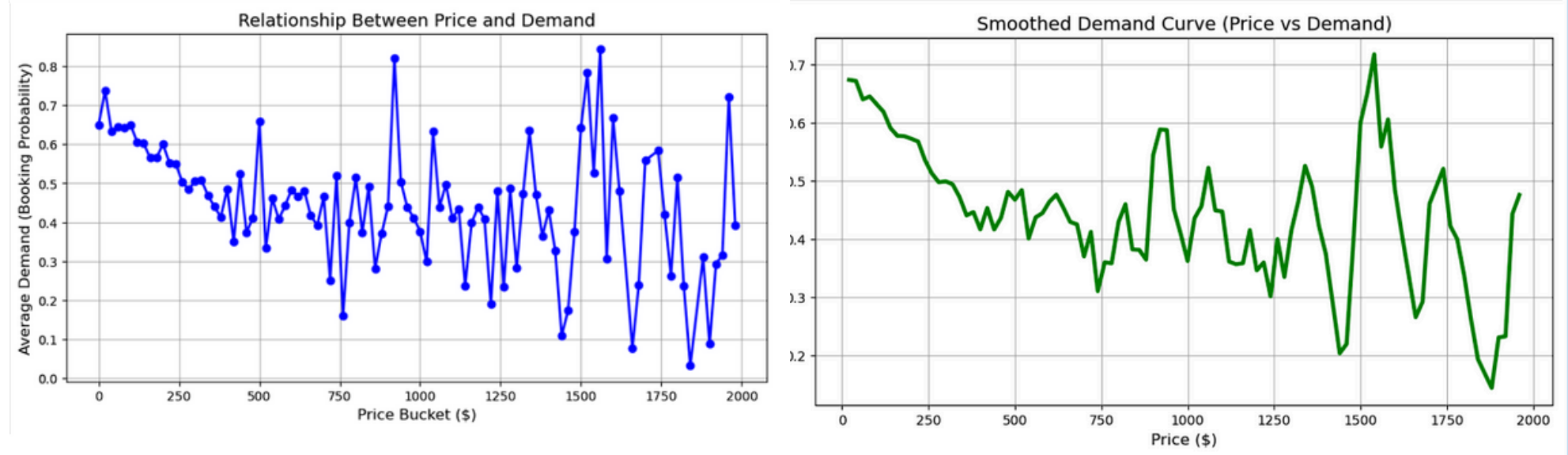
MODEL RESULTS

Estimated elasticity is consistently negative
Confirms the law of demand at scale
Elasticity magnitude shows moderate price sensitivity
Estimates stabilize by ~40% sample size
Scaling out reduces runtime, not elasticity values
Demonstrates efficient, reliable, and interpretable modeling

RMSE: 0.9088771533003887
R²: 7.302840352774176e-06

Sample Size	Elasticity (β) 0-Worker	Elasticity (β) 2 - Workers	Interpretation
5%	-0.24509	-0.24509	Directionally correct but slightly noisy
15%	-0.24513	-0.24513	Elasticity stabilizing
40%	-0.24512	-0.24512	Converged estimate
100%	-0.24513	-0.24513	Benchmark elasticity

Sample Size	Runtime – 0 Worker (sec)	Runtime – 2 Workers (sec/min)	Observation
5%	696.09	606.59	Small data, limited gain
15%	906.27	613.75	Parallelism begins to help
40%	908.78	639.41	Significant speedup
100%	816.18	690.8	Maximum benefit from scaling



QUESTION 3.

PRICE TRENDS (EAST COAST VS WEST COAST)

- **Data Selection and Preprocessing:**

- Data collected for the last 4 quarters (12 months) for 14 cities (6 from the east coast and 8 from the west coast). Used the listings.csv as it provides the most comprehensive information about each listing.
- Preprocess listings.csv: selecting dtypes, fill missing values with mode, mean and median values as necessary, one-hot encoding for property_type and room_type.

```
root
|-- price_numeric: double (nullable = true)
|-- calendar_last_scraped_clean: date (nullable = true)
|-- property_type_clean: string (nullable = true)
|-- room_type_clean: string (nullable = true)
|-- accommodates_clean: integer (nullable = false)
|-- bathrooms_text_clean: string (nullable = true)
|-- bedrooms_clean: integer (nullable = false)
|-- bathrooms_clean: double (nullable = false)
|-- review_scores_rating_clean: integer (nullable = true)
|-- review_scores_cleanliness_clean: integer (nullable = true)
|-- review_scores_location_clean: integer (nullable = true)
|-- host_is_superhost: string (nullable = true)
```

- **Data Exploration results:**

- Prices for the east coast cities have less variance (considering one city at a time) as compared to west coast cities (NYC is an exception!)
- East coast cities have fewer registered airbnb options (likely due to policies for non-primary housing rentals) however, the mean prices are higher along the east coast (Tech and entertainment hubs)

```
+-----+-----+
|price_bucket| count|
+-----+-----+
|      100-199|116805|
|      200-299| 52544|
|      300-499| 38044|
|           500+|216103|
|           <100|102816|
+-----+-----+
```

```
+-----+-----+
|price_bucket| count|
+-----+-----+
|      100-199|140867|
|      200-299| 74777|
|      300-499| 55994|
|           500+|120483|
|           <100| 82168|
+-----+-----+
```

TIME-SERIES ROLLUPS

Time series roll-ups performed for monthly, weekly and daily data.

Weekly roll-ups show the price pattern more distinctly as compared to daily and yearly roll-ups.

We also plot the mean price for monthly, weekly and daily data to visualization purposes.



=== EAST COAST MONTHLY ROLLUP ===						
year	month	count	mean_price	median_price	min_price	max_price
2024	11	23125	215.11697297297297	148.0	8.0	20000.0
2024	12	44987	203.14826505434903	135.0	8.0	20000.0
2025	1	23364	193.9126005820921	130.0	8.0	20000.0
2025	2	23574	193.41804530414865	130.0	8.0	20000.0
2025	3	37734	238.0050087454285	152.0	7.0	238121.0
2025	4	22533	220.45386766076422	143.0	8.0	20000.0
2025	5	22246	231.67342443585363	150.0	8.0	20000.0
2025	6	39324	502.14423761570544	148.0	8.0	50184.0
2025	7	21741	518.9182190331632	150.0	10.0	50083.0
2025	8	21698	441.50391741174303	150.0	3.0	50052.0

=== WEST COAST MONTHLY ROLLUP ===						
year	month	count	mean_price	median_price	min_price	max_price
2024	12	102020	310.3392962164281	175.0	6.0	99999.0
2025	3	98486	324.5637044859168	183.0	1.0	99999.0
2025	4	54	310.037037037037	272.0	85.0	900.0
2025	6	94659	498.3795096081725	185.0	7.0	54280.0
2025	7	10393	1168.4454921581835	225.0	19.0	100000.0
2025	9	99083	600.6752722465004	178.0	8.0	85000.0

ELASTIC NET MODELLING

```
=== PROCESSING CITY: ALBANY ===
```

```
=== TRAINING MODEL: ALBANY ===
```

```
Albany → RMSE: 82.49, MAE: 37.18, R2: 0.474
```

```
=== PROCESSING CITY: BOSTON ===
```

```
=== TRAINING MODEL: BOSTON ===
```

```
Boston → RMSE: 2360.89, MAE: 434.34, R2: 0.333
```

```
=== TRAINING MODEL: EAST ===
```

```
east → RMSE: 2068.44, MAE: 328.16, R2: 0.212
```

```
=== TRAINING MODEL: WEST ===
```

```
west → RMSE: 2376.08, MAE: 459.61, R2: 0.137
```

```
Running pipeline for east_Q1 ...
```

```
=== TRAINING MODEL: EAST_Q1 ===
```

```
east_Q1 → RMSE: 2033.80, MAE: 335.51, R2: 0.241
```

```
Running pipeline for east_Q2 ...
```

```
=== TRAINING MODEL: EAST_Q2 ===
```

```
east_Q2 → RMSE: 1531.49, MAE: 240.94, R2: 0.117
```

```
Running pipeline for west_Q1 ...
```

```
=== TRAINING MODEL: WEST_Q1 ===
```

```
west_Q1 → RMSE: 2579.74, MAE: 485.47, R2: 0.268
```

```
Running pipeline for west_Q2 ...
```

```
=== TRAINING MODEL: WEST_Q2 ===
```

```
west_Q2 → RMSE: 2027.15, MAE: 365.24, R2: 0.130
```

CLUSTER SPECIFICATIONS

Number of Vcpu	% data	Time	R-squared value (east , west)
(1+4) 8 vcpu	[Q4]100%	169.73s	0.212 , 0.137
	[Q2]50%	159.77s	0.117 , 0.130
	[Q1]25%	149.88s	0.241 , 0.268
(1+2) 4 vcpu	[Q4]100%	273.11s	0.212 , 0.136
	[Q2]50%	243.13s	0.118 , 0.130
	[Q1]25%	236.6s	0.243 , 0.269

THANK YOU

Q&A